UNIVERSIDADE PRESBITERIANA MACKENZIE
ENGENHARIA DA COMPUTAÇÃO

PROJETO FINAL - PROGRAMAÇÃO ORIENTADA A OBJETOS COM JAVA

SISTEMA DE AGENDAMENTO E GESTÃO DE CONSULTAS MÉDICAS

CONTEXTO

Sua equipe de desenvolvimento foi contratada para construir uma aplicação web para clínicas e consultórios médicos. O objetivo é permitir que pacientes possam agendar suas consultas online, visualizar seu histórico de consultas. Os profissionais de saúde, por sua vez, devem gerenciar seus horários e manter registros simples de atendimento.

OBJETIVOS DO PROJETO

- Desenvolver um sistema web baseado em Java com Servlets, JSP e HTML/CSS, utilizando o modelo de arquitetura MVC.
- Aplicar os princípios da Programação Orientada a Objetos (POO).
- Utilizar o SQLite como banco de dados relacional para persistência de informações.
- Implementar controle de sessão, validação de usuários e o uso de rotas protegidas com HttpSession.

ESTRUTURA INICIAL FORNECIDA

BANCO DE DADOS FORNECIDO: db.db

Você trabalhará com o banco db.db, que já contém usuários cadastrados. Abaixo está a estrutura básica de tabelas:

Tabela 'usuarios':

id, nome, email, cpf, celular, tipo (paciente, admin, médico), senha,created_at

Tabela 'consultas':

id, paciente_id, profissional_id, data_hora, status, observacoes

Tabela 'prontuarios' (Ficha Clínica):

id, paciente_id, profissional_id, data, anotacoes_medicas, prescricoes

---

UNIVERSIDADE PRESBITERIANA MACKENZIE
ENGENHARIA DA COMPUTAÇÃO

PROJETO MAVEN com a seguinte estrutura:

```
clinica/
├── pom.xml -> Configuração do Maven (plugins e dependências)
├── src/
   └── main/
       ├── java/
       |   └── com/mack/clinica/
       |                 ├── controller/ -> Servlets
       |                 ├── model/      -> Classes de domínio e DAO
       |                 └── util/       -> Utilitários (conexão BD)
       └── webapp/
           ├── login.jsp
           ├── paciente_dashboard.jsp
           ├── admin_dashboard.jsp
           ├── agendar_consulta.jsp
           ├── css/styles.css
           └── WEB-INF/
               ├── web.xml -> Mapeamento dos servlets
               ├── db.db   -> Banco de dados SQLite com estrutura inicial
```

FUNCIONALIDADES QUE DEVERÃO SER DESENVOLVIDAS

A estrutura inicial do sistema inclui a autenticação de usuários, dashboards distintos e agendamento de consultas. A partir dessa base, seu grupo deverá implementar as seguintes funcionalidades:

Funcionalidades para Pacientes:

- Visualizar dados pessoais: Crie uma página acessível via menu "**Meu Cadastro**" onde o paciente poderá visualizar (e futuramente editar) suas informações básicas como nome, email, telefone e CPF.
- Visualizar agendamentos: Crie uma página acessível via menu "**Minha Agenda**", onde o paciente poderá listar as consultas já agendadas, com data, horário e médico responsável.

Funcionalidades para Administradores:

- CRUD de Pacientes: Implemente o cadastro, edição, exclusão e listagem de pacientes. Esta funcionalidade deverá ser acessível via menu "**Cadastro de Pacientes**".

- CRUD de Médicos: Implemente o cadastro, edição, exclusão e listagem de médicos. Esta funcionalidade deverá ser acessível via menu "**Cadastro de Médicos**".
- Consulta da Agenda: Implemente uma página acessível via menu "**Consultar Agenda**" onde o administrador poderá visualizar todas as consultas agendadas, filtrando por paciente, médico ou data.
- Ficha Clínica (Prontuário): No menu "**Ficha Clínica**", crie a estrutura inicial de um formulário simples que poderá futuramente receber informações médicas básicas relacionadas à consulta. (O preenchimento não precisa ser funcional nesta fase).

Importante:

- Todas as operações que envolverem acesso ao banco de dados devem ser feitas por classes DAO específicas.
- As páginas JSP devem seguir a separação do CSS em arquivos externos (já fornecido).
- Toda alteração no banco de dados deve ser refletida no banco SQLite fornecido.

Observações:

- O sistema já faz o redirecionamento correto baseado no tipo de usuário (não é necessário implementar novamente).
- O agendamento de consultas já está implementado e armazenado no banco (não é necessário reimplementar).
- As páginas de menu e logout também já foram configuradas.

ENTREGÁVEIS OBRIGATÓRIOS

- ✓ Projeto funcional entregue via repositório GitHub com histórico de commits do grupo.
- ✓ Precisa me incluir no projeto do Git usuário github: nilton.canto@mackenzie.br
- ✓ Código organizado, comentado e funcional
- ✓ README.md com instruções de execução
- ✓ Apresentação oral com demonstração funcional do projeto

CRITÉRIOS DE AVALIAÇÃO

- ✓ Funcionalidades implementadas: 4.0
- ✓ Organização e clareza do código: 2.0
- ✓ Uso correto de POO: 2.0
- ✓ Documentação e repositório Git: 1.0
- ✓ Apresentação funcional do projeto: 1.0

UNIVERSIDADE PRESBITERIANA MACKENZIE
ENGENHARIA DA COMPUTAÇÃO

TUTORIAL INICIAL: INSTALAÇÃO E CONFIGURAÇÃO DO AMBIENTE

Instalação do Maven

1. Acesse o site oficial:
   https://maven.apache.org/download.cgi
2. Baixe a última versão estável do **Apache Maven**.
3. Extraia o arquivo ZIP em uma pasta de sua escolha, por exemplo: C:\Program Files\Apache\Maven.
4. Configure as variáveis de ambiente:
   - **MAVEN_HOME**: aponte para a pasta onde extraiu o Maven.
   - **Path**: adicione `%MAVEN_HOME%\bin` ao Path do sistema.
5. Verifique a instalação:
   - Abra o terminal (Prompt de Comando ou PowerShell) e digite:

     ```
     mvn -v
     ```
   - Deverá ser exibida a versão do Maven instalada.

Configuração do Visual Studio Code

1. Instale as extensões:
   - **Extension Pack for Java** (by Microsoft)
   - **Maven for Java**
   - **Java Web App - Servlet Support**
2. Certifique-se que o VSCode está apontando corretamente para seu JDK e para o Maven.
3. Para rodar o projeto:
   - Você pode utilizar o terminal interno (Ctrl+ `) e rodar os comandos Maven diretamente.
   - `mvn clean install` Limpa, compila, testa, empacota e instala tudo de novo do zero.
   - `mvn jetty:run`

Opção ao VSCode

- Utilizar o Eclipse IDE com suporte a projetos dinâmicos com Maven.
- Instalar o Apache Tomcat 9 como servidor de aplicações.

UNIVERSIDADE PRESBITERIANA MACKENZIE
ENGENHARIA DA COMPUTAÇÃO

PASSO A PASSO PARA RODAR O PROJETO:

1. **Abrir o projeto** no VSCode (a pasta do projeto mesmo, onde está o pom.xml).
2. **Rodar o comando Maven** no terminal integrado do VSCode: `mvn clean install`
   - Isso vai baixar todas as dependências, compilar o projeto e preparar a pasta target.
   - **Importante**: sem esse comando, o Maven não baixa os JARs e o Jetty não consegue rodar.
3. **Rodar o projeto com o Jetty**: `mvn jetty:run`
   - Isso sobe o servidor Jetty embutido.
   - O site estará disponível em **http://localhost:8080/**.

Cuidados:

- O terminal precisa estar aberto na raiz do projeto (onde está o pom.xml).
- Se der erro falando de **plugin Jetty não encontrado**, normalmente resolve-se com:

  ```
  mvn clean install
  ```
- A primeira vez que rodar pode demorar um pouco porque vai baixar dependências.
- Se mudar algum arquivo .java , tem que **parar** e **executar:**

  ```
  mvn clean install
  mvn jetty:run
  ```

User: admin@mackenzie.br

Senha: admin123

User: james.brown@soul.com

Senha: 123

Obs.: Esses usuários já estão cadastrados no banco de dados inicial db.db.